CSAPP BombLab 实验报告

一、 实验环境

OS: Linux x86_64

工具: GCC, GDB, Objdump

二、 关卡分析与解答

Phae 1：

把%rsi 的内容保存到 callee-saved 寄存器%r12 中，保证不会被修改。同时，%rbp，%rsi 和%rbx 的旧值从低到高(从顶向下）存入了栈。

%rsi 存入 0x1（实际上是%esi 存入 0x1）然后把%rbx 的值存入栈中。

调用<valid_u32_fields>，返回值存在（%rax 低八位）中，然后 test 对返回值逻辑与，结果存到标志位中。

如果返回 1 就是 jne not equal 成立，那么就跳转到 402450.

这种情况下，先用 xor %eax,%eax 起到清空%eax 的作用（因为 not equal，本来里面有值。），然后使用 lea 把当前栈底指针-20 的位置放入%rdx，把十六进制 403182 放入%esi。

在 phase1 的结尾会遇到函数 p1_compute,在 shell 中 disassemble 如下：

```
Dump of assembler code for function p1_compute:
   0x0000000000401fe0 <+0>:     endbr64
   0x0000000000401fe4 <+4>:     push   %rbp
   0x0000000000401fe5 <+5>:     mov    %rdi,%rdx
   0x0000000000401fe8 <+8>:     mov    %rdi,%rax
   0x0000000000401feb <+11>:    shr    $0x20,%rax
   0x0000000000401fef <+15>:    mov    %rdx,%rsi
   0x0000000000401ff2 <+18>:    mov    %eax,%edi
   0x0000000000401ff4 <+20>:    shr    $0x10,%rsi
   0x0000000000401ff8 <+24>:    mov    %rsp,%rbp
   0x0000000000401ffb <+27>:    call   0x401fc0 <p1_step1>
   0x0000000000402000 <+32>:    mov    %edi,%esi
   0x0000000000402002 <+34>:    mov    %edx,%edi
   0x0000000000402004 <+36>:    mov    %eax,%ecx
   0x0000000000402006 <+38>:    call   0x401fb0 <p1_step0>
   0x000000000040200b <+43>:    mov    %ecx,%esi
   0x000000000040200d <+45>:    mov    %eax,%edi
   0x000000000040200f <+47>:    call   0x401fd0 <p1_step2>
   0x0000000000402014 <+52>:    pop    %rbp
   0x0000000000402015 <+53>:    ret
End of assembler dump.
```

分析知道其作用：把传入的第一个参数(视为 x）存在 rax，高 32 位移动到低 32 位；%rsi 的内容为 x>>16.最后传入 p1_step1 的函数参数是:%edi: x >> 32; %rsi: x >> 16.

%eax 返回：return 5(x >> 32) ^ (48929 + (x >> 16 的后 32 位))（32 位值）；

之后%esi:x 的高 32 位；%edi: x 的低 32 位。 %ecx: 5(x >> 32) ^ (48929 + (x >> 16 的后 32

位))

P1_step0 返回(0xa32b ^ x 高 32 位) + 4 * x 的低 32 位.
而后%esi 存%ecx 里的第一轮返回值（记为 r1），第二轮返回值存到%edi（记为 r2）。
P1_step2 调用后，返回(2r1 + r2) ^ 0x9bd1.

我们对其三个 p1 内子函数反汇编：

```
(gdb) disas p1_step0
Dump of assembler code for function p1_step0:
   0x0000000000401fb0 <+0>:     endbr64
   0x0000000000401fb4 <+4>:     xor    $0xa32b,%esi
   0x0000000000401fba <+10>:    lea    (%rsi,%rdi,4),%eax
   0x0000000000401fbd <+13>:    ret
End of assembler dump.
(gdb) disas p1_step1

Dump of assembler code for function p1_step1:
   0x0000000000401fc0 <+0>:     endbr64
   0x0000000000401fc4 <+4>:     lea    (%rdi,%rdi,4),%eax
   0x0000000000401fc7 <+7>:     add    $0xbf21,%esi
   0x0000000000401fcd <+13>:    xor    %esi,%eax
   0x0000000000401fcf <+15>:    ret
End of assembler dump.
```

则说明 p1_step1(x, y) :return 5x ^ (48929 + y)

```
(gdb) disas p1_step2
Dump of assembler code for function p1_step2:
   0x0000000000401fd0 <+0>:     endbr64
   0x0000000000401fd4 <+4>:     lea    (%rdi,%rsi,2),%eax
   0x0000000000401fd7 <+7>:     xor    $0x9bd1,%eax
   0x0000000000401fdc <+12>:    ret
End of assembler dump.
```

返回 (x + 2y) ^ 0x9bd1;
将三个 step 函数和主函数的运算规则写入解码程序，即可得到 solve_phase_1：

```c
int solve_phase_1(uint64_t challenge, char *out, size_t cap) {
    uint32_t values[1] = {0};
    size_t count = 1;
    uint64_t x = challenge;
    uint32_t x_shr = (uint32_t) (x >> 32);

    uint32_t a1 = ((uint32_t)((x >> 16) + 0xbf21)) ^ ((uint32_t)(5 * (x >> 32)));
    uint32_t a2 = (uint32_t)((x_shr ^ 0xa32b) + 4 * (uint32_t)x);
    values[0] = ((uint32_t)(a2 + 2 * a1)) ^ 0x9bd1;
    return write_answer(out, cap, values, count);
}
```

公开列表以外 seed 验证记录：



Phase 2:

我把我的思考过程写在代码旁边：

Challenge 存在

0x0000000000402080 <+0>: endbr64

0x0000000000402084 <+4>: push %rbp

0x0000000000402085 <+5>: mov %rsp,%rbp

0x0000000000402088 <+8>: push %r13

0x000000000040208a <+10>: mov %rsi,%r13

0x000000000040208d <+13>: mov $0x6,%esi //指的是要输入 6 个变量

这说明 rsi 存到了 r13，也就是 challenge 在 r13 上，这是因为 r13 是 callee-saved.

0x0000000000402092 <+18>: push %r12

0x0000000000402094 <+20>: mov %rdi,%r12

Rdi，也就是输入字符串的地址存到 rdi。

0x0000000000402097 <+23>: push %rbx

0x0000000000402098 <+24>: sub $0x28,%rsp

现在 rbp 下:

R13 旧值 8 位

R12 旧值 8 位

Rbx 旧值八位

5 * 8 的空位

0x000000000040209c <+28>: call 0x402bf0 <valid_u32_fields> //

判断输入的六个数字是否合法，如果合法%al 为 1，触发 jne，跳转到 4020b8，否则不跳转。

0x00000000004020a1 <+33>: test %al,%al

0x00000000004020a3 <+35>: jne 0x4020b8 <phase_02+56>

0x00000000004020a5 <+37>: xor %ebx,%ebx

0x00000000004020a7 <+39>: lea -0x18(%rbp),%rsp //舍弃 rsp 下移得到的 5*8 空位

0x00000000004020ab <+43>: mov %ebx,%eax //eax = ebx

0x00000000004020ad <+45>: pop %rbx

0x00000000004020ae <+46>: pop %r12

0x00000000004020b0 <+48>: pop %r13 //三个寄存器恢复原值。

因为这个函数 phase_02 也作为更大的函数的 callee，有保留调用时传入的 callee-saved register 的义务。

0x00000000004020b2 <+50>: pop %rbp //恢复传入时的栈底指针。

0x00000000004020b3 <+51>: ret 返回。

4020a5 后的这一块进入后，函数清除数据，恢复寄存器原值，直接返回，不再做其他操作。

0x00000000004020b4 <+52>: nopl 0x0(%rax)

0x00000000004020b8 <+56>: mov %eax,%ebx

如果 eax = 1，存入 ebx，后面有用。

0x00000000004020ba <+58>: lea -0x2c(%rbp),%rax rbp - 44

把第六个空位中间，第八个空位中间的地址给 rax，rcx

0x00000000004020be <+62>: lea -0x3c(%rbp),%rcx

0x00000000004020c2 <+66>: mov $0x403173,%esi //应该是 sscanf 的格式字符串

0x00000000004020c7 <+71>: push %rax

0x00000000004020c8 <+72>: lea -0x30(%rbp),%rax

把原来的 rbp - 44 的位置存到 stack 中。

把 rbp - 48 的位置给到 rax。

下一页：

--Type <RET> for more, q to quit, c to continue without paging--

0x00000000004020cc <+76>: lea -0x40(%rbp),%rdx //rdx = rbp - 64

0x00000000004020d0 <+80>: mov %r12,%rdi //rdi 恢复。

0x00000000004020d3 <+83>: push %rax //rbp - 48 存到 stack 中

0x00000000004020d4 <+84>: lea -0x34(%rbp),%r9

0x00000000004020d8 <+88>: lea -0x38(%rbp),%r8

0x00000000004020dc <+92>: xor %eax,%eax //clear eax

前面 rdx 是 rbp - 64，说明从 rbp 下第八个的开头开始写入。换句话说，从五个 rsp 留出空位从头写入。

0x00000000004020de <+94>: call 0x4011e0 <__isoc99_sscanf@plt>

0x00000000004020e3 <+99>: pop %rdx //rbp - 48 存入 rdx

0x00000000004020e4 <+100>: pop %rcx //rbp - 44 存入 rcx

0x00000000004020e5 <+101>: cmp $0x6,%eax //是否读取了 6 个数

这说明 defuser 对应的函数需要传入 6 个数。

0x00000000004020e8 <+104>: jne 0x4020a5 <phase_02+37>

如果是读取了 6 个数，执行如下代码：

0x00000000004020ea <+106>: mov %r13,%rax //%r13 challenge 进入 rax

0x00000000004020ed <+109>: mov -0x40(%rbp),%esi //rsi 存 rbp-64 对应地址的值【注意：这里是 mov，不是 leaq】

记 challenge 存值为 x

0x00000000004020f0 <+112>: shr $0x7,%rax x >> 7

0x00000000004020f4 <+116>: add %eax,%eax (uint32_t)(x >> 7) * 2

0x00000000004020f6 <+118>: and $0x1e,%eax 与 0x1e &

0x00000000004020f9 <+121>: add $0x1a,%eax + 0x1a

0x00000000004020fc <+124>: cmp %eax,%esi 这里是与 sscanf 中写入的第一个数字作比较，如果一样就往下，不一样跳转。

0x00000000004020fe <+126>: jne 0x4020a5 <phase_02+37>

0x0000000000402100 <+128>: lea -0x40(%rbp),%rdx rbp - 64 存到 rdx

0x0000000000402104 <+132>: mov $0xf,%edi edi = 15

0x0000000000402109 <+137>: mov $0x10,%ecx ecx = 16

0x000000000040210e <+142>: mov %r13,%rax rax 再次赋予 challenge：x

0x0000000000402111 <+145>: shr %cl,%rax x >> 16

0x0000000000402114 <+148>: and $0x3f,%eax eax = (uint_32t)(x >> 16) & 0x3f

0x0000000000402117 <+151>: add %esi,%eax 把第一个字符（这里其实是 x 前一轮的运算结果）加到 eax 中。

0x0000000000402119 <+153>: mov 0x4(%rdx),%esi rbp - 60 的值移动到 esi

0x000000000040211c <+156>: xor %edi,%eax 15 ^ 当前 eax

0x000000000040211e <+158>: cmp %eax,%esi 如果 eax == 输入的第二个值，那么向下，否则提前返回。

0x0000000000402120 <+160>: jne 0x4020a5 <phase_02+37>

0x0000000000402122 <+162>: add $0x3,%ecx ecx = 19

0x0000000000402125 <+165>: add $0xf,%edi edi = 30

0x0000000000402128 <+168>: add $0x4,%rdx rdx = rbp - 56

0x000000000040212c <+172>: cmp $0x1f,%ecx 31 如果不等于 ecx，回到+142 处。也就是说，这是个循环结构！

0x000000000040212f <+175>: jne 0x40210e <phase_02+142>

如果 ecx = 31，那么就返回整个函数。而且因为此时 ebx 早就设置为 1，返回也不清空 ebx，所以返回值 rax 最终也被设置为 1.也就是说，如果循环能够完整走完，返回值就是 1，这也是判断是否成功的依据。

0x0000000000402131 <+177>: jmp 0x4020a7 <phase_02+39>

第 i 轮： edi = 15*i, ecx = 13 + 3*i

从第二轮开始：（上一轮数据 + (uint_32t)(x >> (13 + 3 * i)) & 0x3f ） ^ （15*i）

根据整体的汇编代码，可以知道，在 solve 函数中按照循环结构，依据数组中上一个元素值生成下一个的值，即可得到符合要求的输出。

```c
int solve_phase_2(uint64_t challenge, char *out, size_t cap) {
    uint32_t values[6] = {0};
    size_t count = 6;
    uint64_t x = challenge;
    uint32_t temp = ((uint32_t)(x >> 7)) * 2;
    values[0] = (temp & 0x1e) + 0x1a;
    int d = 0;
    int c = 13;
    for (int i = 1; i <= 5; i++) {
        d += 15;
        c += 3;
        temp = ((uint32_t)(x >> (13 + 3*i))) & 0x3f;
        temp += values[i - 1];
        temp = temp ^ d;
        values[i] = temp;
    }
    return write_answer(out, cap, values, count);
}
```

公开列表以外 seed 验证记录：



Phase_03:

Dump of assembler code for function phase_03:

0x0000000000402230 <+0>: endbr64

0x0000000000402234 <+4>: push %rbp

0x0000000000402235 <+5>: mov %rsp,%rbp rbp 和 rsp 一个值，建立新的栈底

0x0000000000402238 <+8>: push %r12 记录 r12

0x000000000040223a <+10>: mov %rsi,%r12

0x000000000040223d <+13>: mov $0x2,%esi esi = 2，极有可能指要输入 2 个变量。

0x0000000000402242 <+18>: push %rbx

0x0000000000402243 <+19>: mov %rdi,%rbx

0x0000000000402246 <+22>: sub $0x10,%rsp

此时 rbp 下方有旧的 r12，rbx + 2 * 8 的空间，应该是为了 sscanf 存下读入的两个新值而腾出的。

0x000000000040224a <+26>: call 0x402bf0 <valid_u32_fields>

0x000000000040224f <+31>: test %al,%al //返回 1 = true 表示有效，否则无效。

0x0000000000402251 <+33>: jne 0x402260 <phase_03+48> //有效就跳转

//格式不匹配时：

0x0000000000402253 <+35>: xor %eax,%eax 清空

0x0000000000402255 <+37>: add $0x10,%rsp rsp 抛弃腾出的多余值

0x0000000000402259 <+41>: pop %rbx

0x000000000040225a <+42>: pop %r12

0x000000000040225c <+44>: pop %rbp

Rbx，r12，rbp 恢复原值，然后返回。

0x000000000040225d <+45>: ret

0x000000000040225e <+46>: xchg %ax,%ax

格式匹配时：

0x0000000000402260 <+48>: xor %eax,%eax 清空

0x0000000000402262 <+50>: lea -0x14(%rbp),%rcx rcx 是 2*8 空间第二个的开头

0x0000000000402266 <+54>: lea -0x18(%rbp),%rdx 2*8 空间第一个的开头

0x000000000040226a <+58>: mov $0x40317f,%esi 格式字符串

0x000000000040226f <+63>: mov %rbx,%rdi rdi 恢复为一开始的值。

0x0000000000402272 <+66>: call 0x4011e0 <__isoc99_sscanf@plt>

0x0000000000402277 <+71>: cmp $0x2,%eax

0x000000000040227a <+74>: jne 0x402253 <phase_03+35> //如果成功录入的数字不是 2，错误，不做其他操作直接返回，返回值 0。

同样设 challenge 的值为 x：

0x000000000040227c <+76>: mov %r12d,%edi //%edi 存 x 的低 32 位。(uint32_t)x.

0x000000000040227f <+79>: xor %eax,%eax %eax = 0;

0x0000000000402281 <+81>: and $0x7,%edi & 7

（我一开始没发现一个问题：实际上对于 2*8 的空间，程序只把%rbp - 24,%rbp-20 开头对应的 2*4 的空间填上了值，地址更低的 8bytes 空间没有使用。）

0x0000000000402284 <+84>: cmp %edi,-0x18(%rbp) edi 与 values[0]比较

0x0000000000402287 <+87>: jne 0x402255 <phase_03+37> //不同就直接返回 0，炸弹爆炸。

0x0000000000402289 <+89>: mov %r12,%rax rax = x

0x000000000040228c <+92>: mov %r12,%rsi rsi = x

0x000000000040228f <+95>: shr $0x3,%rax rax: x >> 3

0x0000000000402293 <+99>: shr $0x20,%rsi rsi: x >> 32

0x0000000000402297 <+103>: xor %eax,%esi esi = (uint32_t)((x >> 3) ^ (x >> 32))

0x0000000000402299 <+105>: call 0x4021d0 <p3_dispatch>

传入的首个数是 x & 7,大小 0-7，第二个数是上面的 esi。

0x000000000040229e <+110>: cmp %eax,-0x14(%rbp) 要求返回值和传入数组的第二个值一样，也就是说，第二个值要根据 p3_dispatch 的逻辑算出。

0x00000000004022a1 <+113>: sete %al 如果一样，设置为 1.

0x00000000004022a4 <+116>: jmp 0x402255 <phase_03+37>

Dump of assembler code for function p3_dispatch:

0x00000000004021d0 <+0>: endbr64

0x00000000004021d4 <+4>: mov %edi,%eax

0x00000000004021d6 <+6>: mov %esi,%edi

0x00000000004021d8 <+8>: cmp $0x7,%eax

0x00000000004021db <+11>: ja 0x402225 <p3_dispatch+85>

0x00000000004021dd <+13>: notrack jmp *0x4057e0(,%rax,8)

0x00000000004021e5 <+21>: nopl (%rax)

0x00000000004021e8 <+24>: jmp 0x402140 <p3_case_0>

0x00000000004021ed <+29>: nopl (%rax)

0x00000000004021f0 <+32>: jmp 0x402180 <p3_case_3>

0x00000000004021f2 <+34>: nopw 0x0(%rax,%rax,1)

0x00000000004021f8 <+40>: jmp 0x4021c0 <p3_case_7>

0x00000000004021fa <+42>: nopw 0x0(%rax,%rax,1)

0x0000000000402200 <+48>: jmp 0x4021b0 <p3_case_6>

0x0000000000402202 <+50>: nopw 0x0(%rax,%rax,1)

0x0000000000402208 <+56>: jmp 0x402190 <p3_case_4>

0x000000000040220a <+58>: nopw 0x0(%rax,%rax,1)

0x0000000000402210 <+64>: jmp 0x402170 <p3_case_2>

0x0000000000402215 <+69>: nopl (%rax)

0x0000000000402218 <+72>: jmp 0x4021a0 <p3_case_5>

0x000000000040221a <+74>: nopw 0x0(%rax,%rax,1)

0x0000000000402220 <+80>: jmp 0x402160 <p3_case_1>

0x0000000000402225 <+85>: xor %eax,%eax

0x0000000000402227 <+87>: ret

这是一个 switch 语句，每个语句跳转到一个函数（由于是跳转，下一条地址不压入栈，不会再返回到 p3_dispatch 中。

Switch 语句相关部分：

Dump of assembler code for function p3_dispatch:

0x00000000004021d0 <+0>: endbr64

0x00000000004021d4 <+4>: mov %edi,%eax

0x00000000004021d6 <+6>: mov %esi,%edi

0x00000000004021d8 <+8>: cmp $0x7,%eax

0x00000000004021db <+11>: ja 0x402225 <p3_dispatch+85>

0x00000000004021dd <+13>: notrack jmp *0x4057e0(,%rax,8)

0x00000000004021e5 <+21>: nopl (%rax)

```
   0x00000000004021e8 <+24>:    jmp    0x402140 <p3_case_0>
   0x00000000004021ed <+29>:    nopl   (%rax)
   0x00000000004021f0 <+32>:    jmp    0x402180 <p3_case_3>
   0x00000000004021f2 <+34>:    nopw   0x0(%rax,%rax,1)
   0x00000000004021f8 <+40>:    jmp    0x4021c0 <p3_case_7>
   0x00000000004021fa <+42>:    nopw   0x0(%rax,%rax,1)
   0x0000000000402200 <+48>:    jmp    0x4021b0 <p3_case_6>
   0x0000000000402202 <+50>:    nopw   0x0(%rax,%rax,1)
   0x0000000000402208 <+56>:    jmp    0x402190 <p3_case_4>
   0x000000000040220a <+58>:    nopw   0x0(%rax,%rax,1)
   0x0000000000402210 <+64>:    jmp    0x402170 <p3_case_2>
   0x0000000000402215 <+69>:    nopl   (%rax)
   0x0000000000402218 <+72>:    jmp    0x4021a0 <p3_case_5>
   0x000000000040221a <+74>:    nopw   0x0(%rax,%rax,1)
   0x0000000000402220 <+80>:    jmp    0x402160 <p3_case_1>
   0x0000000000402225 <+85>:    xor    %eax,%eax
   0x0000000000402227 <+87>:    ret
End of assembler dump.
(gdb) y
✕ Undefined command: "y".  Try "help".
(gdb) x/8xg 0x4057e0
0x4057e0:    0x00000000004021f8    0x0000000000402200
0x4057f0:    0x0000000000402208    0x0000000000402210
0x405800:    0x0000000000402218    0x0000000000402220
0x405810:    0x00000000004021e8    0x00000000004021f0
```

说明：0 对应 case7，1 对应 case6，2 对应 4，3 对应 2，4 对应 5，5 对应 1，6 对应 0，7 对应 3.

把关系写成一个数组：int map[8] = {7, 6, 4, 2, 5, 1, 0, 3};

```
(gdb) disassemble p3_case_0
Dump of assembler code for function p3_case_0:
   0x0000000000402140 <+0>:    endbr64
   0x0000000000402144 <+4>:    mov    %edi,%eax
   0x0000000000402146 <+6>:    shr    $0x2,%edi
   0x0000000000402149 <+9>:    xor    $0x976f,%eax
   0x000000000040214e <+14>:   add    %edi,%eax
   0x0000000000402150 <+16>:   ret
```

P3_case_x 语句太多，就不列出来了。都仅几行，并且只以%rdi 内值为参数。通过把各个分支的逻辑写入一个大的 C 辅助函数，再在 solve 函数中加入此函数，即可通关。

公开列表以外 seed 验证记录：



Phase_04:

Dump of assembler code for function phase_04:

0x0000000000402320 <+0>: endbr64

0x0000000000402324 <+4>: push %rbp

0x0000000000402325 <+5>: mov %rsp,%rbp

0x0000000000402328 <+8>: push %r12

0x000000000040232a <+10>: mov %rsi,%r12 //存 r12 旧值，challenge 存入 r12

0x000000000040232d <+13>: mov $0x1,%esi //esi = 1

0x0000000000402332 <+18>: push %rbx

0x0000000000402333 <+19>: mov %rdi,%rbx //写入的字符串存入 rbx

0x0000000000402336 <+22>: sub $0x10,%rsp //空出 2*8 空间。

0x000000000040233a <+26>: call 0x402bf0 <valid_u32_fields>

0x000000000040233f <+31>: test %al,%al

0x0000000000402341 <+33>: jne 0x402350 <phase_04+48>

无格式问题跳转到+48：

（）这一块代表格式有误/其他问题，直接释放 rsp 空出的 16bytes 空间，将寄存器返回旧值，返回。）

（ **0x0000000000402343 <+35>:** add $0x10,%rsp

0x0000000000402347 <+39>: xor %eax,%eax

0x0000000000402349 <+41>: pop %rbx

0x000000000040234a <+42>: pop %r12

0x000000000040234c <+44>: pop %rbp

0x000000000040234d <+45>: ret)

0x000000000040234e <+46>: xchg %ax,%ax

0x0000000000402350 <+48>: xor %eax,%eax //%eax = 0；

0x0000000000402352 <+50>: lea -0x14(%rbp),%rdx //sscanf 用格式字符串得到变

量，存储的位置。

```
   0x0000000000402356 <+54>:    mov     $0x403182,%esi      //格式字符串
   0x000000000040235b <+59>:    mov     %rbx,%rdi           //写入的字符串为收个参
数
   0x000000000040235e <+62>:    call    0x4011e0 <__isoc99_sscanf@plt>
   0x0000000000402363 <+67>:    cmp     $0x1,%eax           //说明 count = 1
   0x0000000000402366 <+70>:    jne     0x402343 <phase_04+35>
设 x = challenge
   0x0000000000402368 <+72>:    mov     %r12,%rcx       %rcx = x
   0x000000000040236b <+75>:    mov     %r12,%rax       %rax = x
   0x000000000040236e <+78>:    mov     %r12d,%edi      x 的低 32 位存入 rdi
   0x0000000000402371 <+81>:    xor     %esi,%esi       esi = 0
   0x0000000000402373 <+83>:    shr     $0x20,%rax      x >> 32
   0x0000000000402377 <+87>:    shr     $0x7,%rcx       x >> 7
   0x000000000040237b <+91>:    and     $0x7f,%edi      edi = ((uint32_t)x) & 0x7f
   0x000000000040237e <+94>:    mov     $0x7f,%edx      edx = 0x7f
   0x0000000000402383 <+99>:    xor     %eax,%ecx       %ecx = (uint32_t)((x >> 32) ^
(x >> 7);
   0x0000000000402385 <+101>:   call    0x4022b0 <p4_walk>
   0x000000000040238a <+106>:   cmp     %eax,-0x14(%rbp)
   0x000000000040238d <+109>:   sete    %al
   0x0000000000402390 <+112>:   add     $0x10,%rsp
   0x0000000000402394 <+116>:   pop     %rbx
   0x0000000000402395 <+117>:   pop     %r12
   0x0000000000402397 <+119>:   pop     %rbp
   0x0000000000402398 <+120>:   ret
End of assembler dump.
```

P4_walk 部分汇编代码:

Edi = a1 esi = a2 ,edx = a3; ecx = a4;

[假设传入的 a3>=a2 推导。]

Dump of assembler code for function p4_walk:

```
   0x00000000004022b0 <+0>:     endbr64
   0x00000000004022b4 <+4>:     mov     %edi,%eax       %eax = a1;
   0x00000000004022b6 <+6>:     mov     %edx,%edi       %edi = a3;
   0x00000000004022b8 <+8>:     sub     %esi,%edi       %edi = a3 - a2;
   0x00000000004022ba <+10>:    shr     $1,%edi         (a3 - a2)>>1;
   0x00000000004022bc <+12>:    add     %esi,%edi       edi = a2 + ((a3 - a2)>>1);
   0x00000000004022be <+14>:    cmp     %eax,%edi       a1 == a2 + ((a3 - a2)>>1) (= (a2
+ a3)/2.)
   0x00000000004022c0 <+16>:    je      0x402310 <p4_walk+96>   equal，则跳转。
```

0x00000000004022c2 <+18>: push %rbp
0x00000000004022c3 <+19>: mov %rsp,%rbp
0x00000000004022c6 <+22>: push %rbx
0x00000000004022c7 <+23>: lea (%rdi,%rcx,1),%ebx ebx = (a2 + a3)/2 + a4;
0x00000000004022ca <+26>: sub $0x8,%rsp 空 8bytes
0x00000000004022ce <+30>: cmp %edi,%eax
0x00000000004022d0 <+32>: jb 0x4022f0 <p4_walk+64> 如果 a1 < (a2 + a3)/2
//到这里，就说明没有跳转，a1 > a2 + ((a3-a2)>>1)
0x00000000004022d2 <+34>: lea 0x1(%rdi),%esi %esi = 左边界 = (a2 + a3)/2 + 1;
0x00000000004022d5 <+37>: mov %eax,%edi %edi = a1; //%edi 保持为 a1，即要被搜寻的对象。
0x00000000004022d7 <+39>: call 0x4022b0 <p4_walk>
递归调用，但是最终递归会完成，这时需要在最上层的 p4_walk 函数中接受其返回值:
0x00000000004022dc <+44>: lea (%rax,%rax,2),%eax %eax = 3 * %eax;
0x00000000004022df <+47>: xor %ebx,%eax (mid + a4) ^ (3 * 缩小边界后的返回值);
0x00000000004022e1 <+49>: mov -0x8(%rbp),%rbx rbx = rbp - 8 中的值，恢复原值
0x00000000004022e5 <+53>: leave
0x00000000004022e6 <+54>: xor $0x1f,%eax eax = eax ^ 0xlf
0x00000000004022e9 <+57>: ret
0x00000000004022ea <+58>: nopw 0x0(%rax,%rax,1)
0x00000000004022f0 <+64>: lea -0x1(%rdi),%edx //a1 < 中间值怎么做。edx 给到(a2 + a3)/2 - 1.
0x00000000004022f3 <+67>: mov %eax,%edi //%edi = a1;
0x00000000004022f5 <+69>: call 0x4022b0 <p4_walk>

0x00000000004022fa <+74>: lea (%rax,%rax,2),%eax
0x00000000004022fd <+77>: xor %ebx,%eax (%edi + a4) ^ (3 * 缩小边界后的返回值);
0x00000000004022ff <+79>: mov -0x8(%rbp),%rbx
0x0000000000402303 <+83>: leave
0x0000000000402304 <+84>: xor $0xe,%eax 这个值还和 0xe 作^.
0x0000000000402307 <+87>: ret
0x0000000000402308 <+88>: nopl 0x0(%rax,%rax,1)
0x0000000000402310 <+96>: lea 0xc0(%rdi),%eax ~~0xc0 + 2*%edi (%rdi 高 32 位已经为 0。)~~
不对，是 lea，不是 add，不是把%rdi(mid) + 0xc0 加到 eax 上，lea 是覆盖%eax 原值的意思，所以现在%eax 的值变成了 0xc0 + %rdi(即 0xc0 + mid).
0x0000000000402316 <+102>: xor %ecx,%eax %eax = %eax ^ %ecx; 和 ecx^
0x0000000000402318 <+104>: ret
最终成功的版本返回的是 a4 ^ (0xc0 + 2 * a1).
但是其实不需要在意成功的版本返回了什么，只需要在意 p4_walk 的调用逻辑。

End of assembler dump.

实际上，可以发现每次 p4_walk 使用(mid + 1, end)或者(st,mid-1)调用得到的值，都会被很复杂地处理（^(mid + a4,而且 a4 不知道说什么），还要与一个很奇怪的常数值作异或。）所以应当可以推断出，对给定的四个数，完全没有简单的闭式解。

故而只能将上述汇编等价于：

```
int F(int a1, int a2, int a3, int a4) {
    int m = a2 + (((unsigned)(a3 - a2)) >> 1);

    if (a1 == m) {
        return (m + 0xc0) ^ a4;
    }

    if (a1 > m) {
        int y = F(a1, m + 1, a3, a4);
        return ((3 * y) ^ (m + a4)) ^ 0x1f;
    } else {
        int y = F(a1, a2, m - 1, a4);
        return ((3 * y) ^ (m + a4)) ^ 0x0e;
    }
}
```

实验文档有说：可以在这个文件中添加辅助函数。因此直接以上述框架添加辅助函数，即得 phase4 的 solve 函数。

公开列表以外 seed 验证记录：



Phase 5：

Dump of assembler code for function phase_05:

0x0000000000402400 <+0>: endbr64

0x0000000000402404 <+4>: push %rbp
0x0000000000402405 <+5>: mov %rsp,%rbp //rbp 回到栈顶，开启新栈帧
0x0000000000402408 <+8>: **push %r12** //challenge 存入 r12
0x000000000040240a <+10>: mov %rsi,%r12
0x000000000040240d <+13>: mov $0x2,%esi // esi = 2.
0x0000000000402412 <+18>: **push %rbx**
0x0000000000402413 <+19>: mov %rdi,%rbx //字符串存入 rbx
0x0000000000402416 <+22>: sub $0x10,%rsp //留出 16bits 空位
0x000000000040241a <+26>: call 0x402bf0 <valid_u32_fields>
0x000000000040241f <+31>: test %al,%al
0x0000000000402421 <+33>: jne 0x402430 <phase_05+48>
0x0000000000402423 <+35>: xor %eax,%eax //清空
0x0000000000402425 <+37>: add $0x10,%rsp //释放栈上分配的空间
0x0000000000402429 <+41>: pop %rbx
0x000000000040242a <+42>: pop %r12
0x000000000040242c <+44>: pop %rbp
0x000000000040242d <+45>: ret //恢复三个寄存器旧值
0x000000000040242e <+46>: xchg %ax,%ax

0x0000000000402430 <+48>: xor %eax,%eax //%eax = 0
0x0000000000402432 <+50>: lea -0x18(%rbp),%rcx //两个数据的存放地点
0x0000000000402436 <+54>: lea -0x1c(%rbp),%rdx
0x000000000040243a <+58>: mov $0x40317f,%esi //格式字符串
0x000000000040243f <+63>: mov %rbx,%rdi
0x0000000000402442 <+66>: call 0x4011e0 <__isoc99_sscanf@plt>
0x0000000000402447 <+71>: cmp $0x2,%eax //说明应该输入 2 个值
0x000000000040244a <+74>: jne 0x402423 <phase_05+35> //清空旧数据，返回 0
设 challenge 存的值为 x
0x000000000040244c <+76>: lea -0x14(%rbp),%rsi rsi = rbp - 0x14，也就是输入的
0x0000000000402450 <+80>: mov %r12,%rdi rdi = x
0x0000000000402453 <+83>: call 0x4023a0 <p5_traverse>
0x0000000000402458 <+88>: mov %eax,%edx edx = 返回值
0x000000000040245a <+90>: xor %eax,%eax eax = 0
0x000000000040245c <+92>: cmp %edx,-0x1c(%rbp) 返回值 == 第一个输入值
0x000000000040245f <+95>: jne 0x402425 <phase_05+37> 如果不是，不再做其他操作，返回 0.
如果是：
0x0000000000402461 <+97>: mov -0x14(%rbp),%eax eax = rbp - 0x14.
0x0000000000402464 <+100>: cmp %eax,-0x18(%rbp) 如 rbp - 0x14 == rbp - 0x18（写入的第二个值），就设置为 1，返回 1.
0x0000000000402467 <+103>: sete %al
0x000000000040246a <+106>: jmp 0x402425 <phase_05+37> //返回，不再改变 rax

值。
End of assembler dump.
a1 和 a2 分别是传入值的第一个和第二个。
可以分析出来，程序会根据 x 的传入值算出一个值填在 rbp - 0x14 b2，然后返回一个值 b1，然后只有 b1 == a1，b2 == a2 时返回 1，所以要让 a1 = b1，a2 = b2。

p5_traverse:
(gdb) disassemble p5_traverse
Dump of assembler code for function p5_traverse:
0x00000000004023a0 <+0>: endbr64
0x00000000004023a4 <+4>: mov %rdi,%rax %rax = x;
0x00000000004023a7 <+7>: mov %rdi,%rcx %rcx = x;
0x00000000004023aa <+10>: mov %rsi,%r8 %r8 存 rbp - 0x14 的地址。
0x00000000004023ad <+13>: and $0x7,%eax %rax = x & 7.
0x00000000004023b0 <+16>: shr $0x3,%rcx %rcx = (x >> 3);
0x00000000004023b4 <+20>: mov 0x405b20(,%rax,8),%rdx %rdx 为某个链表的头指针。
0x00000000004023bc <+28>: mov %rdi,%rax %rax 更新为 x
0x00000000004023bf <+31>: shr $0x20,%rdi %rdi = x >> 32;
0x00000000004023c3 <+35>: shr $0x6,%rax %rax = x>>6;
0x00000000004023c7 <+39>: mov 0x8(%rdx),%esi 这个链表的第二个元素送入%esi。
0x00000000004023ca <+42>: xor %edi,%eax eax = (uint32_t)((x>>32) ^ (x>>6))
0x00000000004023cc <+44>: xor 0xc(%rdx),%eax eax = eax ^ (链表第三个元素)
0x00000000004023cf <+47>: mov %ecx,%edi edi = (uint32_t)(x >> 3)
0aass x00000000004023d1 <+49>: xor $0xe025,%eax eax = eax ^ 0xe025;
0x00000000004023d6 <+54>: mov %esi,(%r8) %esi 的现值给到地址指向处。所以现在 rbp-14 地址上存的其实是链表的第二个元素。
0x00000000004023d9 <+57>: add %esi,%eax eax = eax + esi
0x00000000004023db <+59>: and $0x3,%edi edi = edi & 0x3;
如果是 0 就跳转到 90，代表结束；否则继续向下
0x00000000004023de <+62>: je 0x4023fa <p5_traverse+90>
0x00000000004023e0 <+64>: mov $0x1,%ecx ecx = 1;

//设到这里是 eax = a0
0x00000000004023e5 <+69>: mov (%rdx),%rdx 很可能链表头指针指向的是下一个链表的地址，所以这其实是链表头指针从指向当前变成指向下一个的过程。
0x00000000004023e8 <+72>: add $0x1,%ecx %ecx += 1;
0x00000000004023eb <+75>: mov 0x8(%rdx),%esi %esi = 这个链表的第一个值;
0x00000000004023ee <+78>: xor 0xc(%rdx),%eax eax = (链表第二个值) ^ a0
0x00000000004023f1 <+81>: add %esi,%eax %eax = 链表第一个值+(链表

第二个值) ^ a0.

0x00000000004023f3 <+83>: mov %esi,(%r8) rbp-0x14 存当前指向链表的首个值。

0x00000000004023f6 <+86>: cmp %ecx,%edi 比较 ecx 的值，如果<= (edi = (uint32_t)(x >> 3) & 3)那么就继续。其实就是 ecx = i 的从 1 开始的 for 循环。

0x00000000004023f8 <+88>: jae 0x4023e5 <p5_traverse+69>

0x00000000004023fa <+90>: ret

End of assembler dump.

注意这里需要读取 8 个链表的地址信息，所以我 break *0x402421 在一开头设置了断点，然后借此查找到了 0x405b20 开头的数组里的八个链表首地址：

gef➤ x/8gx 0x405b20

0x405b20 <p5_heads>: 0x0000000000405bd0 0x0000000000405bb0

0x405b30 <p5_heads+16>: 0x0000000000405ba0 0x0000000000405b60

0x405b40 <p5_heads+32>: 0x0000000000405bf0 0x0000000000405c10

0x405b50 <p5_heads+48>: 0x0000000000405b70 0x0000000000405b80

然后用每 4byte 查询内存的方法得到了所有链表的值，存在了 defuser.c 中。

a0 紧挨着 b0，所以每个链表大小是 16，从而猜测:xor 0xc(%rdx),%eax eax = (链表第二个值) ^ a0 这里的第二个值也只能存在 4bites 的地方，是 uint32_t.

故猜测链表元素是这样的：

```
struct node{
    struct node* next;
    uint32_t val1;
    uint32_t val2;
}
```

把整个链表（包括 node 的实现，每个链表的内部值）都加入 defuser.c 后，第五关通过。

完整链表数组：

```
struct node{
    struct node* next;
    uint32_t val1;
    uint32_t val2;
};

struct node node1, node2, node3, node4, node5, node6, node7, node8, node9, node10, node11, node12;

// 3. 按照你的要求
// e node1 ~ node12 (一一对应 0x405b60 ~ 0x405c10)
struct node node1  = { &node10, 0x00000537, 0x00005be8 };  // 0x405b60
struct node node2  = { &node3,  0x0000013f, 0x00008483 };  // 0x405b70
struct node node3  = { &node7,  0x00000626, 0x00004705 };  // 0x405b80
struct node node4  = { NULL,    0x00000825, 0x0000f1fe };  // 0x405b90
struct node node5  = { &node1,  0x00000786, 0x00006e44 };  // 0x405ba0
struct node node6  = { &node5,  0x00000326, 0x00008612 };  // 0x405bb0
struct node node7  = { &node9,  0x00000b89, 0x0000e70a };  // 0x405bc0
struct node node8  = { &node6,  0x00000433, 0x0000b23d };  // 0x405bd0
struct node node9  = { &node11, 0x00000244, 0x0000e41e };  // 0x405be0
struct node node10 = { &node12, 0x0000093b, 0x00004a00 };  // 0x405bf0
struct node node11 = { &node4,  0x0000004e, 0x00003f98 };  // 0x405c00
struct node node12 = { &node2,  0x00000a2a, 0x00009419 };  // 0x405c10
                                                           //
```

公开列表以外 seed 验证记录：



Phase 6：

仍然设 challenge = x

Dump of assembler code for function phase_06:

0x0000000000402540 <+0>: endbr64

0x0000000000402544 <+4>: push %rbp

0x0000000000402545 <+5>: mov %rsp,%rbp //rsp = rbp，开启新栈帧

0x0000000000402548 <+8>: push %r12

0x000000000040254a <+10>: mov %rsi,%r12 // %r12 存 x

0x000000000040254d <+13>: mov $0x1,%esi //esi = 1

0x0000000000402552 <+18>: push %rbx

0x0000000000402553 <+19>: mov %rdi,%rbx //字符串存到 rbx

0x0000000000402556 <+22>: sub $0x10,%rsp 分配 16bytes 空间

0x000000000040255a <+26>: call 0x402bf0 <valid_u32_fields>

0x000000000040255f <+31>: test %al,%al

0x0000000000402561 <+33>: jne 0x402570 <phase_06+48> 如果格式正确（是 1）跳转到 48

【 **0x0000000000402563 <+35>:** add $0x10,%rsp 释放栈上分配的空间

0x0000000000402567 <+39>: xor %eax,%eax //eax = 0

0x0000000000402569 <+41>: pop %rbx

0x000000000040256a <+42>: pop %r12

0x000000000040256c <+44>: pop %rbp 三个寄存器恢复旧值

0x000000000040256d <+45>: ret 返回

0x000000000040256e <+46>: xchg %ax,%ax 】

0x0000000000402570 <+48>: xor %eax,%eax //%eax = 0

0x0000000000402572 <+50>: lea -0x14(%rbp),%rdx //rdx = rbp - 20,也就是分配的 16bytes 空间的最后四个。

```
   0x0000000000402576 <+54>:    mov    $0x403182,%esi        //格式字符串
   0x000000000040257b <+59>:    mov    %rbx,%rdi             //rdi 重新获得字符串
   0x000000000040257e <+62>:    call   0x4011e0 <__isoc99_sscanf@plt>
   0x0000000000402583 <+67>:    cmp    $0x1,%eax             //只需要提取一值
   0x0000000000402586 <+70>:    jne    0x402563 <phase_06+35>   //不是提取到一个值
```
就错了，直接 eax = 0 返回。
```
   0x0000000000402588 <+72>:    mov    %r12,%rdi          //rdi = x
   0x000000000040258b <+75>:    call   0x4024f0 <p6_dispatch>
   0x0000000000402590 <+80>:    cmp    %eax,-0x14(%rbp)     返回值和输入的唯一一个
```
值对比。
```
   0x0000000000402593 <+83>:    sete   %al                  只有相等时才设置为 1
   0x0000000000402596 <+86>:    add    $0x10,%rsp
   0x000000000040259a <+90>:    pop    %rbx
   0x000000000040259b <+91>:    pop    %r12
   0x000000000040259d <+93>:    pop    %rbp
   0x000000000040259e <+94>:    ret
End of assembler dump.
```
相等时返回 1，不相等返回 0，说明需要写入的东西等于 p6_dispatch 的返回值。

```
Dump of assembler code for function p6_dispatch:
   0x00000000004024f0 <+0>:     endbr64
   0x00000000004024f4 <+4>:     push   %rbp
   0x00000000004024f5 <+5>:     mov    %rsp,%rbp          //开启新栈帧
   0x00000000004024f8 <+8>:     push   %rbx
   0x00000000004024f9 <+9>:     mov    %rdi,%rbx          //x  存入%rbx
   0x00000000004024fc <+12>:    shr    $0x6,%rdi          // rdi : x >> 6
   0x0000000000402500 <+16>:    mov    %rbx,%rax          //rax = x
   0x0000000000402503 <+19>:    shr    $0x20,%rax         //%rax = (x >> 32)
   0x0000000000402507 <+23>:    xor    %eax,%edi    //%edi    = (uint32_t)((x >> 32) ^ (x >>
```
6));
```
   0x0000000000402509 <+25>:    mov    %rbx,%rax          //%rax = x;
   0x000000000040250c <+28>:    sub    $0x8,%rsp          //腾出 8bytes 的栈空间
   0x0000000000402510 <+32>:    and    $0x38,%ebx         //uint32_t(0x38 & x);
   0x0000000000402513 <+35>:    and    $0x7,%eax          // eax = uint32_t(0x7 & x);  从
```
而可取 0-7
```
   0x0000000000402516 <+38>:    call   *0x405dc0(,%rax,8)  跳转到函数跳转表，从中有 8
```
个不同函数指针，对应 8 个不同函数。
```
   0x000000000040251d <+45>:    xor    $0x28bb,%eax          //%eax = (返回值) ^ 0x28bb
   0x0000000000402522 <+50>:    mov    %eax,%edi             //%edi =       (返回值) ^
```
0x28bb
```
   0x0000000000402524 <+52>:    call   *0x405dc0(%rbx)       //调用  (0x405dc0 + %rbx)的
```
函数指针
```
   0x000000000040252a <+58>:    mov    -0x8(%rbp),%rbx       //栈空间回收
   0x000000000040252e <+62>:    leave                        //rsp = rbp,rbp 回到旧值
```

```
   0x000000000040252f <+63>:    add    $0x3572,%eax          //eax += 0x3572,然后返回。
   0x0000000000402534 <+68>:    ret
End of assembler dump.
```

0x405dc0 开头的容量为 8 的 8 字节数组：

```
gef➤  x/8gx 0x405dc0
0x405dc0 <p6_ops>:          0x00000000004024d0          0x00000000004024c0
0x405dd0 <p6_ops+16>:       0x00000000004024b0          0x00000000004024d0
0x405de0 <p6_ops+32>:       0x0000000000402490          0x00000000004024e0
0x405df0 <p6_ops+48>:       0x0000000000402470          0x0000000000402490
```

对 8 个函数指针指向的函数分别 disassemble：

均设传入参数为 a：实际上发现传入参数第一步都会被当作 32 位，所以直接对 a uint32_t 化即可。

[0]

```
Dump of assembler code for function p6_op_4:
   0x00000000004024d0 <+0>:     endbr64
   0x00000000004024d4 <+4>:     mov    %edi,%eax      //eax = a
   0x00000000004024d6 <+6>:     ror    $0x2,%eax       //eax  循环右移 2
   0x00000000004024d9 <+9>:     ret
End of assembler dump.
```

[1]

```
Dump of assembler code for function p6_op_3:
   0x00000000004024c0 <+0>:     endbr64
   0x00000000004024c4 <+4>:     mov    %edi,%eax
   0x00000000004024c6 <+6>:     xor    $0x115c,%eax
   0x00000000004024cb <+11>:    lea    (%rax,%rdi,8),%eax
   0x00000000004024ce <+14>:    ret
End of assembler dump.
```

返回 ((a ^ 0x115c) + 8 * a);

[2]

```
Dump of assembler code for function p6_op_2:
   0x00000000004024b0 <+0>:     endbr64
   0x00000000004024b4 <+4>:     lea    0xea6e(%rdi),%eax
   0x00000000004024ba <+10>:    shr    $1,%edi
   0x00000000004024bc <+12>:    xor    %edi,%eax
   0x00000000004024be <+14>:    ret
End of assembler dump.
```

返回 (a + 0xea6e) ^ (a >> 1);

[3]

```
gef➤  disassemble 0x00000000004024d0
Dump of assembler code for function p6_op_4:
   0x00000000004024d0 <+0>:     endbr64
   0x00000000004024d4 <+4>:     mov    %edi,%eax
   0x00000000004024d6 <+6>:     ror    $0x2,%eax
   0x00000000004024d9 <+9>:     ret
End of assembler dump.
```

[0 和 3 一样]

[4]

```
Dump of assembler code for function p6_op_1:
   0x0000000000402490 <+0>:     endbr64
   0x0000000000402494 <+4>:     mov    %edi,%eax
   0x0000000000402496 <+6>:     lea    (%rdi,%rdi,2),%edx      //%edx = (uint32_t)(3*a)
   0x0000000000402499 <+9>:     shl    $0x7,%eax               //%eax = ((uint32_t)a << 7)
   0x000000000040249c <+12>:    add    $0x5ebd,%eax            //%eax = 0x5ebd + (a << 7)
   0x00000000004024a1 <+17>:    xor    %edx,%eax
   0x00000000004024a3 <+19>:    ret
End of assembler dump.
```

最终%eax = (0x5ebd + (a << 7)) ^ (3*a);

[5]

```
Dump of assembler code for function p6_op_5:
   0x00000000004024e0 <+0>:     endbr64
   0x00000000004024e4 <+4>:     mov    %edi,%eax
   0x00000000004024e6 <+6>:     rol    $0x5,%eax
   0x00000000004024e9 <+9>:     xor    $0xffe7,%eax
   0x00000000004024ee <+14>:    ret
End of assembler dump.
```

返回(a 循环左移 5) ^ 0xffe7;

[6]

```
Dump of assembler code for function p6_op_0:
   0x0000000000402470 <+0>:     endbr64
   0x0000000000402474 <+4>:     mov    %edi,%eax
   0x0000000000402476 <+6>:     shr    $0x2,%eax
   0x0000000000402479 <+9>:     xor    %edi,%eax
   0x000000000040247b <+11>:    add    $0xe2a3,%eax
   0x0000000000402480 <+16>:    ret
End of assembler dump.
```

((a >> 2)^a) + 0xe2a3

[7]

Dump of assembler code for function p6_op_1:

```
   0x0000000000402490 <+0>:     endbr64
   0x0000000000402494 <+4>:     mov    %edi,%eax            //%eax = a
   0x0000000000402496 <+6>:     lea    (%rdi,%rdi,2),%edx   //%edx = 3 * a
   0x0000000000402499 <+9>:     shl    $0x7,%eax            // (%eax = (a << 7))
   0x000000000040249c <+12>:    add    $0x5ebd,%eax         // %eax += 0x5ebd
   0x00000000004024a1 <+17>:    xor    %edx,%eax            // %eax = (3*a) ^ ((a << 7) + 0x5ebd)
   0x00000000004024a3 <+19>:    ret
```

End of assembler dump.

把上述 8 个函数的执行逻辑写入一个大的 switch 过程中，然后再写入 solve_phase_6,即得到答案：

```c
uint32_t func_ptr(uint32_t id, uint32_t a) {
    uint32_t temp;
    switch (id){
        case 0:
        case 3:
            return ((a >> 2) | (a << 30));
        case 1:
            return ((a ^ 0x115c) + 8 * a);
        case 2:
            return (a + 0xea6e) ^ (a >> 1);
        case 4:
        case 7:
            return (0x5ebd + (a << 7)) ^ (3*a);
        case 5:
            temp = (a << 5) | (a >> 27);
            return temp ^ 0xffe7;
        case 6:
            return ((a >> 2)^a) + 0xe2a3;
    }
    return -1;
}
int solve_phase_6(uint64_t challenge, char *out, size_t cap) {
    uint32_t values[1] = {0};
    size_t count = 1;
    uint64_t x = challenge;
    uint32_t ebx = (uint32_t)(0x38 & x);
    uint32_t edi = (uint32_t)((x >> 32) ^ (x >> 6));
    uint32_t res; //result
    res = func_ptr(0x7 & x, edi);
    edi = res ^ 0x28bb;
    res = func_ptr(ebx / 8, edi);
    res += 0x3572;
    values[0] = res;
    return write_answer(out, cap, values, count);
}
```

公开列表以外 seed 验证记录：

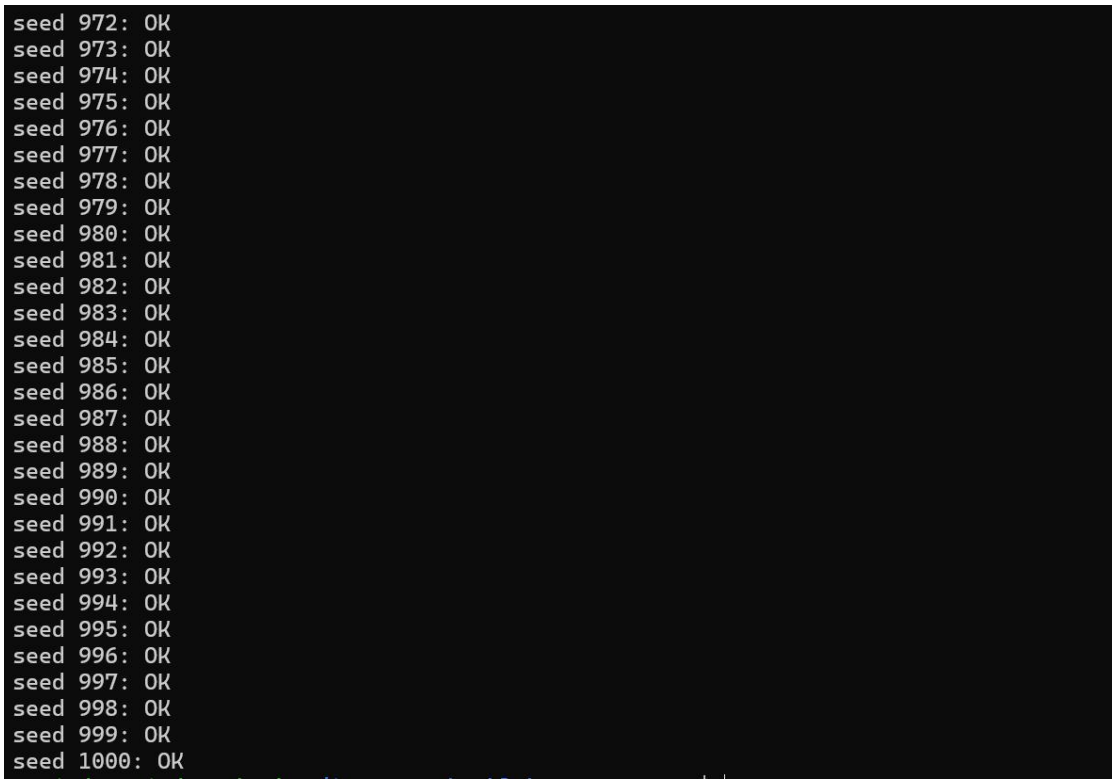


额外验证过程：

将六个汇编代码的流程分析对应的 C 语言写入 defuser.c 后，我尝试了 1000 个种子（包括了大部分在公开列表里的 seed），均显示”OK”，说明我的代码应该是正确的。

```
crmind@crmindnotebook:~/ics-2026-bomblab_25300180031$ for s in $(seq 1 1000); do
    # 把输出存到变量里
    out=$(./defuser --seed "$s" | ./bomb --seed "$s" 2>&1)

    # 只有明确看到 Congratulations 才算成功
    if echo "$out" | grep -q "Congratulations"; then
        echo "seed $s: OK"
    else
        echo "seed $s: FAIL"
        # 如果失败了，把失败的输出保存到文件方便你排查
        echo "$out" > "fail_$s.txt"
    fi
done
seed 1: OK
seed 2: OK
seed 3: OK
seed 4: OK
seed 5: OK
seed 6: OK
seed 7: OK
seed 8: OK
seed 9: OK
seed 10: OK
```



Honor 关卡：

Dump of assembler code for function secret_phase:

0x00000000004025c0 <+0>: endbr64

0x00000000004025c4 <+4>: push %rbp

0x00000000004025c5 <+5>: mov %rsp,%rbp //打开新栈帧

0x00000000004025c8 <+8>: push %r13

设第一个参数 x，第二个 y

0x00000000004025ca <+10>: mov %rdx,%r13 //第二个参数 y 给 r13

0x00000000004025cd <+13>: push %r12

0x00000000004025cf <+15>: mov %rsi,%r12 //第一个参数 x 给%r12

0x00000000004025d2 <+18>: mov $0x3,%esi //esi = 3

0x00000000004025d7 <+23>: push %rbx

0x00000000004025d8 <+24>: mov %rdi,%rbx //输入字符串给%rbx;

0x00000000004025db <+27>: sub $0x18,%rsp //rsp 减小 24，给出 24bytes 空位

0x00000000004025df <+31>: call 0x402bf0 <valid_u32_fields>

0x00000000004025e4 <+36>: test %al,%al

0x00000000004025e6 <+38>: jne 0x402600 <secret_phase+64> //如果不是 0 跳转

如果是 0 如下：

0x00000000004025e8 <+40>: xor %edx,%edx edx 清空

0x00000000004025ea <+42>: add $0x18,%rsp 增加的栈上空位恢复原状

0x00000000004025ee <+46>: mov %edx,%eax

0x00000000004025f0 <+48>: pop %rbx

0x00000000004025f1 <+49>: pop %r12

0x00000000004025f3 <+51>: pop %r13

0x00000000004025f5 <+53>: pop %rbp

0x00000000004025f6 <+54>: ret

0x00000000004025f7 <+55>: nopw 0x0(%rax,%rax,1) //无用，为了内存对齐用的

0x0000000000402600 <+64>: xor %eax,%eax //%eax = 0

此时 rbp 下面是 r13，r12，rbx，24bytes 的空间。

0x0000000000402602 <+66>: lea -0x24(%rbp),%r8 //%r8 是 24bytes 空间的倒数第三个的首地址

0x0000000000402606 <+70>: lea -0x28(%rbp),%rcx //%rcx = 倒数第四个的地址

0x000000000040260a <+74>: mov $0x40317c,%esi //%esi = 0x40317c

0x000000000040260f <+79>: lea -0x2c(%rbp),%rdx //%rdx = 倒数第五个的首地址

0x0000000000402613 <+83>: mov %rbx,%rdi //输入字符串给%rdi

0x0000000000402616 <+86>: call 0x4011e0 <__isoc99_sscanf@plt>

0x000000000040261b <+91>: cmp $0x3,%eax 要录入三个变量

0x000000000040261e <+94>: jne 0x4025e8 <secret_phase+40>

记(uint32_t)y = y0

0x0000000000402620 <+96>: mov %r13d,%ecx %ecx = (uint32_t)y

0x0000000000402623 <+99>: xor %edx,%edx %edx = 0;

0x0000000000402625 <+101>: and $0x3,%ecx %ecx = 3 & y0

0x0000000000402628 <+104>: cmp %ecx,-0x2c(%rbp) 字符串上第一个录入的 value == 3 & y0

0x000000000040262b <+107>: jne 0x4025ea <secret_phase+42> //不是，进入 secret_phase+42

0x000000000040262d <+109>: mov %r12,%rsi //%rsi = x;

[%r9d 的信息]0x0000000000402630 <+112>: imul $0xa4e0,%ecx,%r9d //%r9d = (3 & y0) * 0xa4e0

0x0000000000402637 <+119>: mov %r13,%r8 //%r8 = y;

0x000000000040263a <+122>: shr $0x20,%rsi %rsi = (x >> 32);

0x000000000040263e <+126>: shr $0x20,%r8 //(y >> 32)

记 x 的低 32 位为 x0

0x0000000000402642 <+130>: xor %r12d,%esi //%esi = x0 ^ (uint32_t)(x >> 32);

0x0000000000402645 <+133>: xor %r13d,%r8d //%r8d = (uint32_t)((y >> 32) ^ y);

0x0000000000402648 <+136>: shr $0x10,%r13 //%r13d = (uint32_t)(y >> 16);

0x000000000040264c <+140>: mov %esi,%edi //%edi = x0 ^ (uint32_t)(x >> 32);

0x000000000040264e <+142>: xor %r13d,%r9d //%r9d = ((uint32_t)(y >> 16)) ^ ((3 & y0) * 0xa4e0);

0x0000000000402651 <+145>: xor %r8d,%edi //%edi = (x0 ^ (uint32_t)(x >> 32)) ^ (uint32_t)((y >> 32) ^ y);

0x0000000000402654 <+148>: call 0x4025a0 <secret_mix>

设返回值为 a0.

0x0000000000402659 <+153>: xor %r9d,%eax // %eax = a0 ^ %r9d;

0x000000000040265c <+156>: cmp %eax,-0x28(%rbp) 第二个录入的是否是当前的%eax 值.

x0 ^ (uint32_t)(x >> 32)记为 mix(x):

```
uint32_t mix(uint64_t x){
    return uint32_t(x ^ (x >> 32));
}
```

0x000000000040265f <+159>: jne 0x4025ea <secret_phase+42> //如果不是，返回 0.

0x0000000000402661 <+161>: shr $0x3,%esi //%esi = (mix(x) >> 3);

0x0000000000402664 <+164>: lea 0xae1(%r8,%rcx,1),%edx //%edx = 0xae1 + mix(y) + (3 & y0);

0x000000000040266c <+172>: add %esi,%edx //两个值加起来

0x000000000040266e <+174>: xor %edx,%eax %eax ^= %eax;

0x0000000000402670 <+176>: cmp -0x24(%rbp),%eax 检查字符串第一个录入的值是否是当前的%eax 值。

0x0000000000402673 <+179>: sete %dl 如果是，设置%rdx 低八位位 1，也就是%rdx = 1.但是 edx 的 9-32 位不变！！!要注意

0x0000000000402676 <+182>: jmp 0x4025ea <secret_phase+42> //然后跳转到+42，实际上作用就是返回%rdx 的值.

End of assembler dump.

Secret_mix 函数：

gef➤ disassemble secret_mix

Dump of assembler code for function secret_mix:

0x00000000004025a0 <+0>: endbr64

0x00000000004025a4 <+4>: mov %edi,%eax //%eax = 参数 1;

0x00000000004025a6 <+6>: rol $0x3,%eax // 参数 1 循环左移 3 格

0x00000000004025a9 <+9>: add $0xa4e0,%eax

0x00000000004025ae <+14>: xor $0xae1,%eax

0x00000000004025b3 <+19>: ret

把循环左移三格的%eax 再加 0xa4e0,然后与 0xae1 ^.

End of assembler dump.

gef➤ disassemble secret_copy

Dump of assembler code for function secret_copy:

0x0000000000402680 <+0>: endbr64

0x0000000000402684 <+4>: xor %eax,%eax //eax 清空为 0

0x0000000000402686 <+6>: cs nopw 0x0(%rax,%rax,1) //无用，占位命令

0x0000000000402690 <+16>: movzbl (%rsi,%rax,1),%edx //%edx = %rsi + %rax

注意这一条的内容：把一个字节读出来[地址%rax + arg2（参数 2）]，扩展成 32 位，放到目标寄存器。

0x0000000000402694 <+20>: mov %dl,(%rdi,%rax,1) //把刚刚放入目标寄存器的前八位值放到地址[%rdi + %rax]中。

0x0000000000402697 <+23>: add $0x1,%rax //%rax += 1;

0x000000000040269b <+27>: test %dl,%dl

0x000000000040269d <+29>: jne 0x402690 <secret_copy+16> //如果值不是 0，就重新用增加了 1 的%rax 值重新跳转到+16，进行第二次读取。

0x000000000040269f <+31>: ret

End of assembler dump.

gef➤ disassemble secret_gate

Dump of assembler code for function secret_gate:

设：参数 1 为 str，参数 2 为 x，参数 3 为 y。

0x00000000004026a0 <+0>: endbr64

0x00000000004026a4 <+4>: push %rbp

0x00000000004026a5 <+5>: xor %eax,%eax //%eax = 0

0x00000000004026a7 <+7>: mov $0x14,%ecx //%ecx = 20

0x00000000004026ac <+12>: mov %rsp,%rbp //开启新栈帧

0x00000000004026af <+15>: push %r14

0x00000000004026b1 <+17>: mov %rdx,%r14 //%r14 = y;

0x00000000004026b4 <+20>: push %r13

0x00000000004026b6 <+22>: mov %rsi,%r13 //%r13 = x;

0x00000000004026b9 <+25>: push %r12

0x00000000004026bb <+27>: lea -0xd0(%rbp),%r12 //%r12 = %rbp - 13 * 16;

0x00000000004026c2 <+34>: push %rbx

0x00000000004026c3 <+35>: mov %rdi,%rbx %rbx = str 字符串;

0x00000000004026c6 <+38>: mov %r12,%rdi %rdi = %rbp - 13 * 16;

0x00000000004026c9 <+41>: sub $0xb0,%rsp %rsp 减小 11 *16 格，开出 192bytes 空间，正好到达%rbp - 13 * 16 处。所以实际上%rsp 腾出了 176bytes 的空间。

0x00000000004026d0 <+48>: rep stos %rax,(%rdi) 把从 rdi 开始 8 * 20 的内存都重置为 0.（也就是 16 * 10 的内存，这一步可能是因为栈有概率残留内容，所以清空栈上的残留内容，保证程序安全运行。）

0x00000000004026d3 <+51>: movl $0x0,(%rdi) 将%rdi 地址上的值设置为 0.

0x00000000004026d9 <+57>: mov %rbx,%rdi %rdi = 字符串 str（应该是字符串指针）

0x00000000004026dc <+60>: call 0x401190 <strlen@plt>

0x00000000004026e1 <+65>: sub $0x24,%rax %rax = strlen(str) - 36

0x00000000004026e5 <+69>: cmp $0x7f,%rax %rax == 127

0x00000000004026e9 <+73>: jbe 0x402700 <secret_gate+96> <= 就去+96 处.

如果 strlen(str) > 163:

0x00000000004026eb <+75>: add $0xb0,%rsp 把分配的 11 * 16 的栈空间回收

0x00000000004026f2 <+82>: xor %eax,%eax eax = 0 返回。

0x00000000004026f4 <+84>: pop %rbx

0x00000000004026f5 <+85>: pop %r12

0x00000000004026f7 <+87>: pop %r13

0x00000000004026f9 <+89>: pop %r14

0x00000000004026fb <+91>: pop %rbp

0x00000000004026fc <+92>: ret

0x00000000004026fd <+93>: nopl (%rax)

0x0000000000402700 <+96>: mov %rbx,%rsi //%rsi = 字符串 str

0x0000000000402703 <+99>: mov %r12,%rdi %r12 = %rbp - 13 * 16.

0x0000000000402706 <+102>: call 0x402680 <secret_copy>

其实就是把字符串 str 中以每一个字节每一个字节地移动到以%rdi 开头的另一块地方，也就是移动到栈上，直到遍历到 str 的结尾，才停止移动。

0x000000000040270b <+107>: cmpl $0x4e45504f,-0xb0(%rbp) // 把 %rbp 与 0x4e45504f 比较

0x0000000000402715 <+117>: jne 0x4026eb <secret_gate+75> 不一样就到 75 处返回 0.

这也就是说，str 的 32bytes 末尾位一定要是 0x4e45504f 但是要注意是按照小端序读取这个字符串的 33-36bytes 的，所以低字节对应低位，字节地址从低到高：4f 50 45 4e. 'OPEN'

另外从这里以及后面的代码可以看出字符串第 0-31 位虽然会被读取，但并不会用来校对，所以 0-31 位取任何值都可以。（不能取\0，这样会提前终止字符串复制。）

如果值一样，这样做：

0x0000000000402717 <+119>: mov %r14,%rdx y 移回%rdx

0x000000000040271a <+122>: mov %r13,%rsi x 移回%rsi

0x000000000040271d <+125>: lea -0xac(%rbp),%rdi %rdi = %rbp - 0xac = %rbp - 172.也就是 str 开头后的第 36 位的值，恰好是 NEPO 后的所有字符。

0x0000000000402724 <+132>: call 0x4025c0 <secret_phase>

0x0000000000402729 <+137>: add $0xb0,%rsp 回收栈空间，通过下文可以知道，返回 secret_phase 给出的返回值。

0x0000000000402730 <+144>: pop %rbx

0x0000000000402731 <+145>: pop %r12

0x0000000000402733 <+147>: pop %r13

0x0000000000402735 <+149>: pop %r14

0x0000000000402737 <+151>: pop %rbp

0x0000000000402738 <+152>: ret

End of assembler dump.

综合上述代码，可以看出在 honor 关，bomb 先调用 secret_gate,读取一个字符串的 32-35 位，以及将第 36 位之后的字符切成三个值，检查这三个值是否和通过 challenge 和 context 算出的预期值相符，如果符合，则判通过。

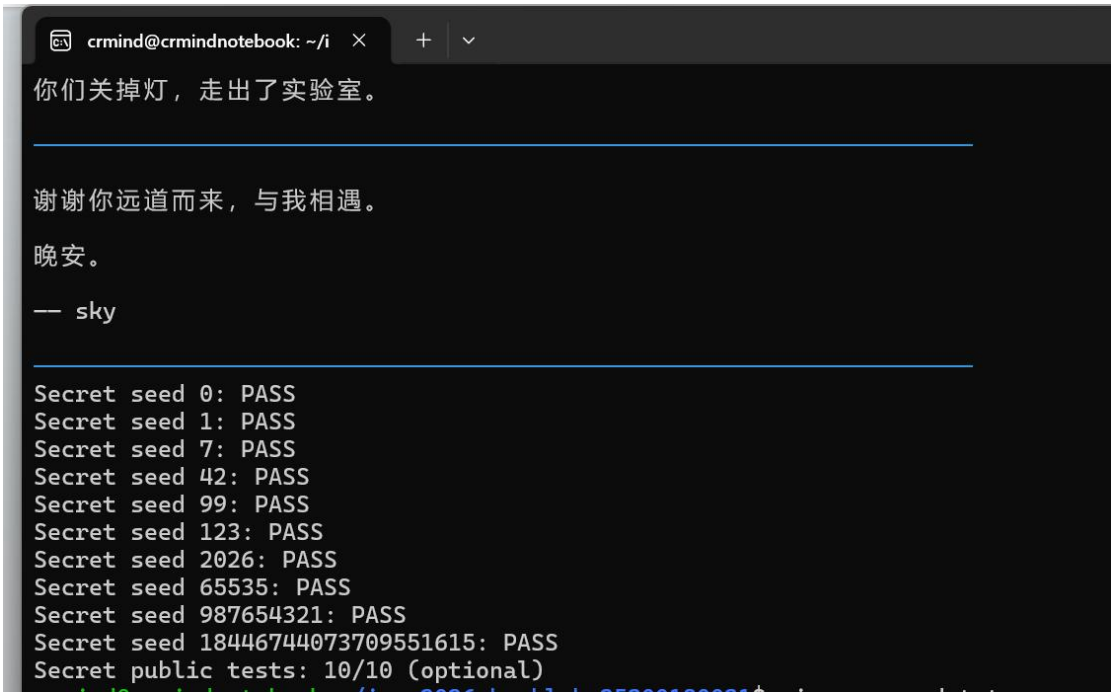
代码实现如下：（计算出三个值的期望值的具体方法在 secret_phase 中已经进行了解析，这里不再重述思考过程。）

```c
uint32_t mix(uint64_t x){
    return (uint32_t)(x ^ (x >> 32));
}
int solve_secret(uint64_t challenge, uint64_t context, char *out, size_t cap) {
    char str0[37] = "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAOPEN";
    str0[35] = 'N';
    uint32_t values[3] = {0};
    //size_t count = 3;
    uint64_t x = challenge;
    uint64_t y = context;
    //uint32_t x0 = (uint32_t)x;
    uint32_t y0 = (uint32_t)y;
    values[0] = 3 & y0;
    uint32_t arg1 = mix(x) ^ mix(y);  //其实是secret_mix的arg1
    uint32_t temp = (((arg1 << 3) | (arg1 >> 29)) + 0xa4e0) ^ 0xae1;
    values[1] = temp ^ (((uint32_t)(y >> 16)) ^ ((3 & y0) * 0xa4e0));
    values[2] = values[1] ^ ((mix(x) >> 3) + 0xae1 + mix(y) + (3 & y0));

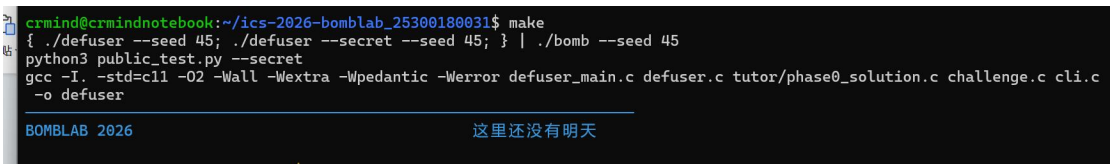
    //拼接：
    size_t len = snprintf(out, cap, "%s%u %u %u", str0, values[0], values[1], values[2]);
    if (len <= 0 || len > cap) return 1;
    return 0;
}
```

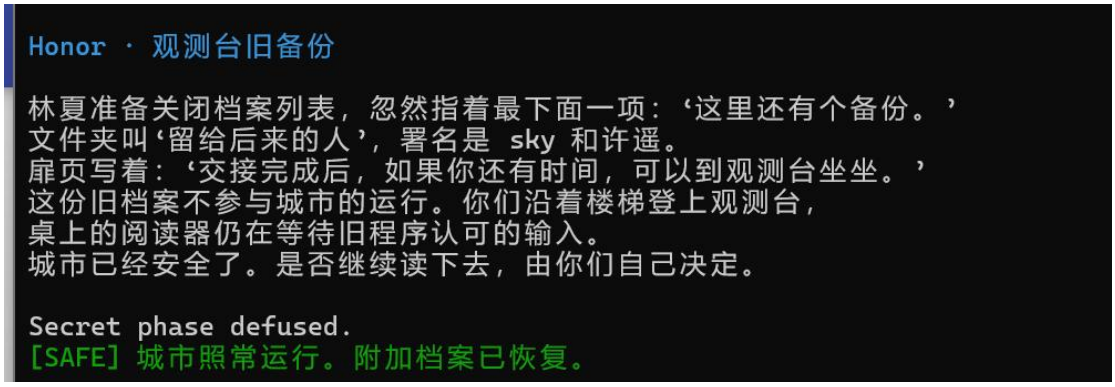
这里由于格式和前六关不同，我单独使用了 snprintf 方法修改了 out 指向的字符串，使之符合格式要求。

本地测试（所有公开 seed，包括对 secret phase 的验证）全部跑通：

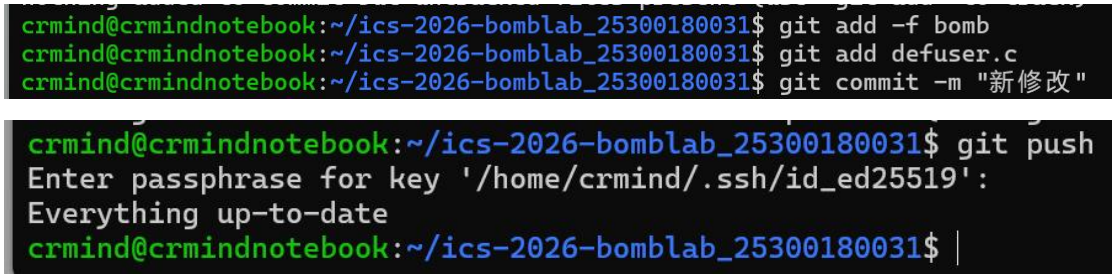


使用非公开 seed 的验证记录：





显示通过。



三、 参考资料与辅助工具说明

1. 引用的资料

• 《深入理解计算机系统》（Computer Systems: A Programmer's Perspective, CSAPP），Randal E. Bryant 等著。

• 课程提供的 BombLab 实验指导书及配套教学文档（tutor/ 目录）。

2. 课程允许的辅助工具的用途

• 使用工具：DeepSeek（网页聊天版本）。

• 具体用途说明：

1. 辅助理解汇编逻辑：在破解第 6 关（solve_phase_6）时，针对其中复杂的函数指针表等汇编指令，使用 DeepSeek 辅助分析了寄存器的跳转逻辑和控制流。

2. 排查内存与指针逻辑错误：在编写第 5 关和第 6 关的 defuser.c 时遇到了指针相关的逻辑错误。起初，我直接读取了 ./bomb 运行时的链表头指针地址并写死在解法中，但未意识到 defuser 和 bomb 是独立运行的两个进程，解法程序无法直接读取炸弹内部的内存数据。我利用 DeepSeek 协助分析了该问题的原因，随后自行修改了代码，改为在解法程序中直接完整录入对应的节点结构（包括录入所有节点的值），最终成功通关。

• 声明：以上工具仅作为辅助理解汇编逻辑和排查异常现象使用，报告中展示的计算关系均已通过 GDB 实际调试验证，核心解法代码均由本人独立编写完成。